



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Music Artists & Albums Public Perception

Workshop #4 — Interactive Dashboards with Plotly Dash

Systems Engineering Curriculum
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Course: Programming for Data Analysis

Professor: Carlos Andres Sierra Virguez

May 30th

Carlos Andres Celis Herrera	20222020051
Juan Diego Lozada Gonzalez	20222020014
Cristian Santiago Lopez Cadena	20222020027

Contents

1	Executive Summary	2
2	Integration Adjustments	2
2.1	Adjustment 1 — Gold DAG: Monthly Partitioning and Append-with-Deduplication .	3
2.2	Adjustment 2 — Gold Schemas: Added <code>ingest_date</code> Field	4
2.3	Adjustment 3 — Dashboard Utilities: Cross-Partition Data Loader	5
2.4	Adjustment 4 — Silver Layer: Single <code>_current</code> File Strategy	6
3	Governance Dashboard Documentation	8
3.1	Layout Description	8
3.2	Component List	9
3.3	KPI Mapping	9
3.4	Design Rationale	10
4	Storytelling Dashboard Documentation	10
4.1	Layout Description and Narrative Flow	10
4.2	Component List	11
4.3	User Story Mapping	11
4.4	Narrative Rationale	11
5	Visualization Design Decisions	12
5.1	Chart Type Selections	12
5.2	Color Palette	12
5.3	Layout and Accessibility Choices	13
6	Reproducibility Evidence	13
6.1	Repository Structure	13
6.2	Setup and Execution Instructions	14
6.3	requirements.txt — Dashboard Dependencies	14
6.4	DAG Execution Evidence	15
6.5	Dashboard Screenshots	15
7	Challenges and Resolutions	19
7.1	Challenge 1 — Gold File Path Mismatch Between DAG and Dashboards	19
7.2	Challenge 2 — Schema Mismatch: <code>ingest_date</code> Field Not Present in Existing Parquets	20
7.3	Challenge 3 — Callback Firing on Every Interval Tick	20
7.4	Challenge 4 — Keyword Noise from Bot Accounts and Platform Names	20

1 Executive Summary

Workshop #4 completes the Medallion Architecture pipeline for the *Music Artists & Albums Public Perception* project by delivering the user-facing presentation layer: two independent, fully interactive web dashboards built with Plotly Dash and powered exclusively by the Gold Parquet files produced in Workshop #3.

The work delivered in this workshop covers four main areas:

- **Integration adjustments** — Three targeted changes to the Gold DAG (`dag_gold.py`) to support monthly partitioning, append-with-deduplication semantics, and a new `ingest_date` field required by the dashboards.
- **Governance Dashboard** (`dashboard/governance_app.py`, port 8050) — A technical-audience dashboard displaying data quality KPIs through metric cards, null-rate and outlier-rate charts, volume comparisons, text-length analysis, and a fully filterable KPI table.
- **Storytelling Dashboard** (`dashboard/storytelling_app.py`, port 8051) — A functional-user dashboard presenting sentiment analysis results, temporal trends, top keywords, comment-type distribution, volume activity, and a cross-source artist comparison.
- **Shared utilities** (`dashboard/utils.py`) — A common module providing data-loading logic, a consistent color palette, and helper functions used by both dashboards.

Both dashboards auto-detect changes in `datalake_gold/` every 15 seconds and re-render only when the Gold files change, ensuring the display always reflects the latest pipeline output without requiring manual restarts.

Key analytical results visible in the dashboards:

- **67.98%** of Reddit comments are positive (avg VADER compound: +0.621); only **14.04%** are negative.
- Top keywords — *album*, *music*, *good* — cluster around positive sentiment, confirming genuine community enthusiasm rather than superficial positivity.
- **BTS** leads the Last.fm chart with approximately 6.8 million unique listeners and 4.12 billion total plays.
- **100%** schema compliance across all three Silver sources; governance dashboard confirms zero null rate on all required fields.

2 Integration Adjustments

This section documents the changes made to the Gold DAG and shared infrastructure specifically in Workshop #4, distinguishing them from the pipeline improvements already covered in Workshop #3. The complete updated architecture is illustrated in Figure 1.

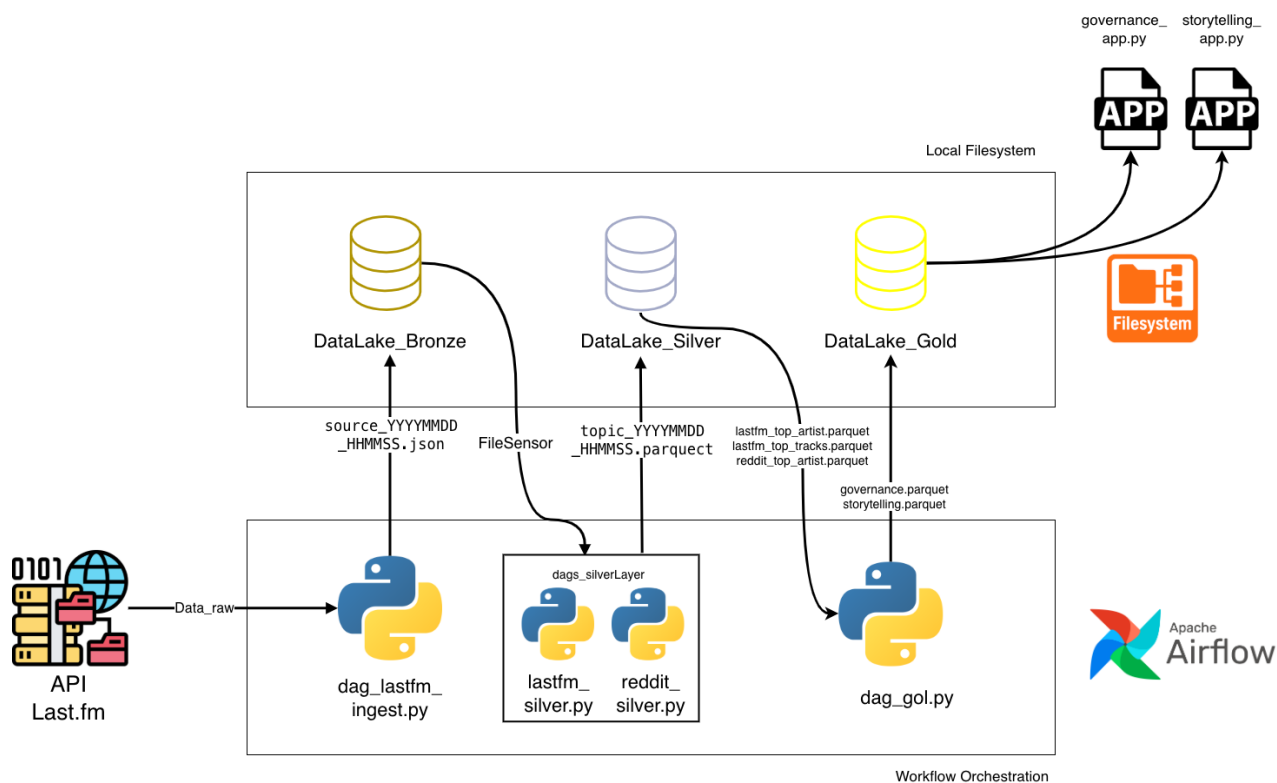


Figure 1: Updated end-to-end Medallion Architecture.

2.1 Adjustment 1 — Gold DAG: Monthly Partitioning and Append-with-Deduplication

Observation

The original `_save_parquet()` function in Workshop #3 wrote a single flat file named `governance.parquet` and `storytelling.parquet`, overwriting it on every weekly run. This design discarded all historical snapshots, preventing cross-week trend analysis in the dashboards.

Change Applied

The persistence function was redesigned to write into a monthly partition structure and accumulate snapshots within the current month using append-with-deduplication semantics:

Listing 1: Monthly partitioned persistence in `dag_gold.py`

```

1 def _save_parquet(rows, schema, base_folder, name):
2     now = datetime.utcnow()
3     year, month = now.year, now.month
4
5     partition_dir = os.path.join(
6         base_folder, name, f"year={year}", f"month={month:02d}"
7     )
8     os.makedirs(partition_dir, exist_ok=True)
9     filepath = os.path.join(partition_dir, f"{name}.parquet")
10
11     new_df = pd.DataFrame(rows, columns=[f.name for f in schema])

```

```

12
13     if os.path.exists(filepath):
14         existing_df = pq.read_table(filepath).to_pandas()
15         combined = pd.concat([existing_df, new_df], ignore_index=True)
16         # Dedup: one snapshot per dimension combination per day
17         dim_cols = [f.name for f in schema
18                     if f.name not in ("computed_at", "ingest_date")]
19         combined["_dedup_date"] = (
20             pd.to_datetime(combined["computed_at"]).dt.date.astype(str)
21         )
22         combined = combined.drop_duplicates(
23             subset=dim_cols + ["_dedup_date"], keep="last"
24         ).drop(columns=["_dedup_date"]).reset_index(drop=True)
25     ...

```

Justification

The partition structure `datalake_gold/<name>/year=YYYY/month=MM/<name>.parquet` mirrors standard data lake conventions and allows the `utils.py` loader to reconstruct the full history by globbing across partitions. Monthly boundaries ensure that previous months are never mutated; only the current-month partition is read and rewritten on each weekly run.

2.2 Adjustment 2 — Gold Schemas: Added ingest_date Field

Observation

Both `GOVERNANCE_SCHEMA` and `STORYTELLING_SCHEMA` lacked a field representing the most recent Silver ingestion date for a given data point. The dashboards required this field to display the *data last-updated* timestamp and to enable temporal filtering in the governance table.

Change Applied

Listing 2: Schema update: `ingest_date` added to both Gold schemas

```

1 GOVERNANCE_SCHEMA = pa.schema([
2     pa.field("source",      pa.string(),   nullable=False),
3     pa.field("field_name",  pa.string(),   nullable=False),
4     pa.field("kpi_type",    pa.string(),   nullable=False),
5     pa.field("value",       pa.float64(),  nullable=False),
6     pa.field("unit",        pa.string(),   nullable=False),
7     pa.field("ingest_date", pa.string(),   nullable=False), # NEW
8     pa.field("computed_at", pa.string(),   nullable=False),
9 ])
10
11 STORYTELLING_SCHEMA = pa.schema([
12     pa.field("metric_type", pa.string(),   nullable=False),
13     pa.field("dim1",         pa.string(),   nullable=False),
14     pa.field("dim2",         pa.string(),   nullable=True),
15     pa.field("record_count", pa.int64(),    nullable=False),
16     pa.field("pct",          pa.float64(),  nullable=False),
17     pa.field("avg_score",    pa.float64(),  nullable=False),
18     pa.field("ingest_date",  pa.string(),   nullable=False), # NEW
19     pa.field("computed_at",  pa.string(),   nullable=False),

```

20])

Justification

Separating `ingest_date` (when the Silver data was last ingested) from `computed_at` (when the Gold computation ran) gives the dashboards precise data provenance information. The governance table header and the narrative card in the storytelling dashboard both surface `ingest_date` to the user.

2.3 Adjustment 3 — Dashboard Utilities: Cross-Partition Data Loader

Observation

The dashboards needed a robust loading function that could transparently handle the new partitioned folder structure, a backward-compatibility rename for early CSV exports, and a fallback path for local development without a running Gold pipeline.

Change Applied

A dedicated `_load_gold_dataset()` function was implemented in `dashboard/utils.py`:

Listing 3: Cross-partition loader in `dashboard/utils.py`

```

1 def _load_gold_dataset(name: str) -> pd.DataFrame:
2     pattern = os.path.join(
3         GOLD_DIR, name, "year=*", "month=*", f"{name}.parquet"
4     )
5     files = sorted(glob.glob(pattern))
6
7     if files:
8         dfs = [pd.read_parquet(f) for f in files]
9         df = pd.concat(dfs, ignore_index=True)
10        # Backward-compat rename
11        if "data_date" in df.columns and "ingest_date" not in df.columns:
12            df = df.rename(columns={"data_date": "ingest_date"})
13        if "computed_at" in df.columns:
14            df = df.sort_values("computed_at").reset_index(drop=True)
15        return df
16
17    # Fallback: CSV export for development
18    csv = os.path.join(NB_DIR, f"export_{name}.csv")
19    if os.path.exists(csv):
20        return pd.read_csv(csv)
21    return pd.DataFrame()

```

Justification

Centralising all data-access logic in `utils.py` ensures both dashboards share identical loading behaviour. The glob pattern `year=*/month=*` traverses all historical partitions in a single call, making the full time-series available to every chart without requiring the dashboard code to know the partition scheme.

Table 1 summarises all three adjustments.

Table 1: Summary of integration adjustments — Workshop #3 → Workshop #4

Component	Before	After
Gold persistence	Single flat <code>_current.parquet</code> overwritten each run.	Monthly partitions <code>year=YYYY/month=MM/</code> ; append-with-dedup within month.
Gold schemas	6 fields; no ingestion date.	7 fields; <code>ingest_date</code> added to both governance and storytelling schemas.
Dashboard loader	Direct <code>pd.read_parquet()</code> on a fixed path.	Cross-partition glob; backward-compat rename; CSV fallback for development.

2.4 Adjustment 4 — Silver Layer: Single `_current` File Strategy

Context

During Workshop #3, the Silver DAGs (`lastfm_silver` and `reddit_silver`) were designed to write output files using a timestamp-based naming convention, producing one Parquet file per execution:

```
datalake_silver/
  lastfm_top_artists/
    lastfm_top_artists_20260528_232836.parquet
    lastfm_top_artists_20260529_011204.parquet
    lastfm_top_artists_20260530_005106.parquet
  lastfm_top_tracks/
    lastfm_top_tracks_20260528_232836.parquet
  ...
```

While this approach mirrored the Bronze accumulation pattern, it raised a series of concerns during the Workshop #4 integration review that led the team to adopt a different strategy for this layer.

Change Applied

The Silver DAGs were updated to write a single overwriting file per source, named `_current`:

```
datalake_silver/
  lastfm_top_artists/
    lastfm_top_artists_current.parquet
  lastfm_top_tracks/
    lastfm_top_tracks_current.parquet
  reddit/
    reddit_music_opinions_current.parquet
```

The naming change is a single-line modification in the persistence call of each Silver DAG:

Listing 4: Silver persistence: timestamp replaced by fixed `_current` filename

```
1 # Before      accumulates one file per execution
2 filepath_out = _save_parquet(
3     df, ARTISTS_SCHEMA, SILVER_ARTISTS_PATH,
4     f"lastfm_top_artists_{timestamp}.parquet"
5 )
```

```
6
7 # After      single overwriting file per source
8 filepath_out = _save_parquet(
9     df, ARTISTS_SCHEMA, SILVER_ARTISTS_PATH,
10     "lastfm_top_artists_current.parquet"
11 )
```

Justification

The decision was based on three technical and architectural arguments:

1. Silver is a transition layer, not a historical record. The Medallion Architecture assigns each layer a distinct responsibility. Bronze is the immutable raw archive — every API response is preserved with a timestamp and never overwritten. Silver's sole responsibility is to provide a clean, normalised, and schema-enforced view of the most recent Bronze data for downstream consumption. Accumulating historical snapshots in Silver would duplicate the archival function already fulfilled by Bronze, without adding analytical value at this layer.

2. Temporal analysis is a Gold responsibility. Historical trend analysis — how listeners, playcount, or sentiment evolved over time — is performed in the Gold DAG using PySpark aggregations over the full Bronze history that Silver already consolidates. Storing multiple versions of the same cleaned dataset in Silver would force the Gold DAG to perform deduplication before every aggregation, increasing both processing time and the risk of logic errors. With a single `_current` file, PySpark reads a clean, deduplicated input on every Gold run.

3. Storage and processing efficiency. A @weekly Silver run over a one-year period would accumulate 52 Parquet files per source under the timestamp strategy. Reading all of them on every Gold execution would increase I/O linearly, with no benefit since the cleaning logic applied by Silver is deterministic — the same Bronze input always produces the same Silver output. The `_current` strategy caps Silver storage at three files regardless of pipeline age (one per source), and reduces Gold's read time to a constant.

Recoverability. A common concern with overwriting strategies is data loss. In this architecture, recoverability is fully guaranteed by Bronze: if a Silver file is lost or corrupted for any reason, the Silver DAG can be re-triggered at any time to regenerate it from the complete Bronze history. The operation is idempotent and takes only the time required for a single Silver execution.

Trade-offs Acknowledged

The team acknowledges that this design sacrifices the ability to audit intermediate cleaning states over time. If the Silver cleaning logic changes between executions, the previous cleaned version is lost. This trade-off was considered acceptable because:

- All cleaning logic is version-controlled in the DAG source code.
- The raw data is always recoverable from Bronze.

- The pipeline is not operating in a regulated environment where intermediate audit trails are a compliance requirement.

Table 2 summarises the two strategies evaluated.

Table 2: Silver persistence strategy comparison		
Criterion	Timestamp accumulation	_current overwrite (adopted)
Storage growth	Linear with executions (52 files/year per source).	Constant (1 file per source).
Gold read cost	Increases with each new file; dedup required.	Constant; clean single read.
Historical auditing	Intermediate cleaned states preserved.	Not preserved; recoverable from Bronze.
Recoverability	Re-run Silver from any Bronze snapshot.	Re-run Silver from full Bronze history.
Complexity	Higher; Gold must handle multiple files per source.	Lower; Gold always reads one file per source.

3 Governance Dashboard Documentation

3.1 Layout Description

The governance dashboard is a single-page Plotly Dash application structured in five visual zones, arranged top-to-bottom following a *summary-to-detail* reading pattern:

1. **Header bar** — Project title, topic angle, team identification, manual refresh button, and last-updated timestamp.
2. **KPI card row** — Four metric cards displaying the most critical governance figures at a glance.
3. **Null rate + Outlier rate row** — Two side-by-side charts covering data completeness and distributional anomalies.
4. **Volume + Text length row** — Two side-by-side charts covering record counts and text field statistics.
5. **Full KPI table** — A paginated, sortable, and filterable DataTable exposing all governance rows.

3.2 Component List

Table 3: Governance dashboard component inventory			
Component	Chart Type	Data Mapping	
Reddit Comments card	Metric card	kpi_type = volume, source = reddit	
Last.fm Artists card	Metric card	kpi_type = volume, source = lastfm_artists	
Schema Compliance card	Metric card	kpi_type = schema_compliance (mean across sources)	
Highest Null Rate card	Metric card	Max value in kpi_type = null_rate; traffic-light color	
Null Rate per Field	Horizontal bar	kpi_type = null_rate; continuous color scale green→red	
Outlier Rate (IQR)	Horizontal bar	kpi_type = outlier_rate; colored by source	
Records by Source	Vertical bar	kpi_type = volume; colored by source	
Mean Text Length	Horizontal bar	kpi_type = text_len_mean; colored by source	
KPI Table	DataTable	All rows; filterable, sortable, paginated (15 rows/page)	

3.3 KPI Mapping

All nine KPI types defined in Workshop #3 are surfaced in the dashboard. Table 4 maps each KPI type to its visual representation and the audience insight it supports.

Table 4: KPI type to visual component mapping		
KPI Type	Visual	Insight
volume	Metric card + bar	Total records ingested per source; detects pipeline failures (zero records).
null_rate	Horizontal bar + table	Completeness per field; red highlighting at >5%.
schema_compliance	Metric card	Single percentage confirming all required fields non-null.
outlier_rate	Horizontal bar	Distributional anomalies per numeric field (IQR ×1.5).
text_len_mean	Horizontal bar	Average text field length; validates that cleaning reduces noise.
text_len_median	Table only	Robust length measure; visible in full KPI table.
text_len_min	Table only	Detects empty comments after cleaning (minimum should be ≥2).
text_len_max	Table only	Flags abnormally long comments (raw max = 3,200 chars).
ingestion_days	Table only	Number of distinct ingestion dates per source.

3.4 Design Rationale

- **Dash Bootstrap Components (FLATLY theme)** was chosen for its clean, whitespace-generous aesthetic suitable for a technical audience and for its responsive grid system, which keeps the layout legible at 1080p without horizontal scrolling.
- **Traffic-light KPI card accents** (green / amber / red via `severity_color()`) give the on-call engineer an immediate pass/fail signal before reading any numbers.
- **Continuous color scale** on the null-rate chart (green→amber→red) communicates severity gradient more efficiently than categorical coloring with only three bins.
- **Auto-refresh via `dcc.Interval`** (15-second poll) with a `gold_state_key()` fingerprint prevents unnecessary re-renders: the dashboard re-draws only when the Parquet files change, not on every tick.

4 Storytelling Dashboard Documentation

4.1 Layout Description and Narrative Flow

The storytelling dashboard is designed for **music label analysts and artist managers** — the functional user defined in Workshop #1. The layout follows a deliberate narrative arc from high-level conclusion to supporting detail:

1. **Header** — Dashboard title, data source description, refresh control.
2. **Narrative Summary Card** — A dynamically generated plain-language insight drawn from live Gold data; the first thing the user reads.
3. **Sentiment Overview** — Donut chart (distribution) side-by-side with trend line (evolution over time).
4. **Conversation Analysis** — Top keywords colored by sentiment + comment type breakdown.
5. **Activity and Source Comparison** — Volume over time (all sources) + Last.fm vs. Reddit artist cross-reference.

Each zone answers a progressively more specific question: *Is the reaction positive?* → *Is it improving?* → *What terms define it?* → *Which artists generate it?*

4.2 Component List

Table 5: Storytelling dashboard component inventory

Component	Chart Type	Data Mapping
Narrative Summary Card	Text card	Dynamically assembled from <code>sentiment_dist</code> , <code>top_keyword</code> , <code>top_artist_lastfm</code>
Community Sentiment	Donut chart	<code>metric_type = sentiment_dist</code> ; green/red/grey fill
Sentiment Trend	Line + fill	<code>metric_type = sentiment_trend</code> ; positive/negative zones shaded
Top Keywords	Horizontal bar	<code>metric_type = top_keyword</code> ; colored by <code>avg_score</code> sentiment bucket
Comment Type Breakdown	Horizontal bar	<code>metric_type = comment_type_dist</code> ; percentage view
Volume Activity	Grouped bar	<code>metric_type = volume_trend</code> ; grouped by source
Artist Cross-reference	Bar + scatter	<code>top_artist_lastfm</code> (listeners) + <code>reddit_artist</code> (mentions overlay)

4.3 User Story Mapping

Table 6: Component to user story mapping

Component	User Story
Narrative card + Donut	“As an analyst, I want to know whether the community reacts positively or negatively to new releases to prioritise communication campaigns.”
Sentiment Trend line	“As a manager, I want to track whether sentiment improves or deteriorates over time to adjust the release strategy.”
Top Keywords	“As an analyst, I want to identify which terms dominate the conversation to understand what aspects of a release generate the most reaction.”
Comment Type Breakdown	“As an analyst, I want to distinguish recommendations from opinions to decide whether the dataset suits content recommendation or sentiment analysis.”
Volume Activity	“As an analyst, I want to see volume over time to identify activity peaks and confirm ingestion regularity.”
Artist Cross-reference	“As a manager, I want to compare quantitative streaming popularity against qualitative Reddit perception to target promotion efforts.”

4.4 Narrative Rationale

The narrative card is generated at render time by querying the live Gold data, ensuring it always reflects the current pipeline output. As visible in Figure 5, it surfaces three numbers (positive

percentage: 68.0%, negative percentage: 14.0%, average compound score: +0.621) alongside the top three keywords (*album*, *music*, *good*) and the leading Last.fm artist (Coldplay, $\approx$ 9M unique listeners in the summary card, 18M in the full cross-reference chart). Plain-language phrasing (“community reacts positively”, “unique listeners”) deliberately avoids technical jargon per the Workshop #4 functional-user guidelines.

5 Visualization Design Decisions

5.1 Chart Type Selections

Table 7: Chart type justification per visualization		
Visualization	Type	Justification
Null rate per field	Horizontal bar	More than 3 categories; horizontal orientation accommodates long field names without truncation. Continuous color scale communicates severity gradient.
Outlier rate	Horizontal bar	Same reasoning; source color distinguishes Reddit vs. Last.fm.
Records by source	Vertical bar	Three categories only; vertical bars communicate volume magnitude more intuitively.
Sentiment distribution	Donut chart	Exactly three categories (positive / neutral / negative); donut variant adds a center annotation with total count for immediate context.
Sentiment trend	Line + fill	Time series; shaded fill emphasizes the magnitude of positive/negative zones; <code>hrect</code> annotation zones make the ± 0.05 thresholds visible without chart clutter.
Top keywords	Horizontal bar	Many categories (≤ 20); colored by sentiment bucket rather than frequency, adding a second dimension of insight within a single chart.
Comment types	Horizontal bar	Four categories; preferred over pie per Workshop #4 guidelines for comparisons with more than three categories.
Volume activity	Grouped bar	Multiple sources on the same time axis; grouped layout prevents stacking from hiding individual source behavior.
Artist cross-reference	Bar + scatter overlay	Primary metric (Last.fm listeners) as bars; Reddit mentions as a scaled diamond-marker scatter, allowing direct visual comparison across two incommensurable scales.

5.2 Color Palette

A single color palette is defined in `dashboard/utils.py` and shared by both applications. Table 8 documents the full palette.

Table 8: Shared color palette (`dashboard/utils.py`)

Token	Hex	Usage
positive	#16a34a	Positive sentiment (green — WCAG AA compliant on white)
negative	#dc2626	Negative sentiment (red — WCAG AA compliant on white)
neutral	#6b7280	Neutral sentiment (grey)
primary	#1e40af	Dashboard headers, borders, primary accent
reddit	#5b9bd5	Reddit source (blue)
lastfm_artists	#70ad47	Last.fm Artists source (green)
lastfm_tracks	#ffc000	Last.fm Tracks source (amber)
background	#f0f4f8	Page background (light grey-blue)

The palette was chosen to be:

- **Colorblind-safe:** the positive/negative green-red pair maintains sufficient contrast under deuteranopia simulation when combined with the neutral grey anchor, and the Reddit/Last.fm blue-green-amber triad is distinguishable under all common forms of color vision deficiency.
- **Semantically consistent:** green always means “good” or “positive” across both dashboards; red always means “bad” or “negative”. Source colors are identical in every chart where multiple sources appear.
- **Minimally varied:** using the same token set in two independent applications prevents palette drift as both evolve independently.

5.3 Layout and Accessibility Choices

- **No horizontal scrolling:** all charts use `dbc.Container(fluid=True)` with Bootstrap’s 12-column grid. The widest row (null-rate chart at `md=7`) fits within 1080p viewports without wrapping.
- **Consistent font:** Inter (via Plotly layout `font` parameter) is used across all chart text for visual coherence with the Bootstrap body font.
- **Source annotations:** every chart includes a source label via axis title, legend, or hover template. The header of each dashboard specifies the data sources (Reddit + Last.fm).
- **Disabled mode bar:** `config={"displayModeBar": False}` removes the Plotly toolbar from all charts, keeping the interface clean for non-technical users in the storytelling dashboard.

6 Reproducibility Evidence

6.1 Repository Structure

The final repository structure relevant to Workshop #4 is:

```
music-pipeline/
|-- dashboard/
|   |-- governance_app.py          # Governance dashboard (port 8050)
|   |-- storytelling_app.py       # Storytelling dashboard (port 8051)
|   '-- utils.py                  # Shared palette, loaders, helpers
|-- dags/
|   |-- dag_lastfm_ingest.py
|   |-- dags_silverLayer/
|       |-- lastfm_silver.py
|       '-- reddit_silver.py
|   '-- dag_gold.py                # Updated in Workshop #4
|-- datalake_bronze/
|-- datalake_silver/
|-- datalake_gold/                 # Partitioned: governance/ storytelling/
|-- notebooks/
|-- requirements.txt
|-- Dockerfile
|-- docker-compose.yml
'-- README.md
```

6.2 Setup and Execution Instructions

The complete pipeline can be reproduced from a fresh clone using the following steps:

```
# 1. Clone and enter the repository
git clone <repository-url>
cd music-pipeline

# 2. Start Airflow (builds the custom image on first run)
docker compose up -d

# 3. Wait for the webserver to be healthy, then trigger DAGs in order:
#   Airflow UI -> http://localhost:8080 (user: airflow / pass: airflow)
#   Trigger: lastfm_ingest -> lastfm_silver -> reddit_silver -> gold_pipeline

# 4. Install dashboard dependencies (outside Docker, in a local venv)
pip install dash dash-bootstrap-components plotly pandas pyarrow

# 5. Launch dashboards (two separate terminals)
python dashboard/governance_app.py      # -> http://localhost:8050
python dashboard/storytelling_app.py    # -> http://localhost:8051
```

6.3 requirements.txt — Dashboard Dependencies

The following packages were added to `requirements.txt` for the dashboard layer:

```
dash>=2.14.0
dash-bootstrap-components>=1.5.0
plotly>=5.18.0
pandas>=2.0.0
pyarrow>=18.0.0
requests>=2.32.5
beautifulsoup4>=4.14.3
```

6.4 DAG Execution Evidence

Figure 2 shows the Airflow UI at `localhost:8080` with all four DAGs active and in a successful state. The `gold_pipeline` DAG shows 3 successful runs tagged with `gold`, `governance`, `pyspark`, and `storytelling`; `lastfm_ingest` accumulates 17 daily runs; `lastfm_silver` shows 7 weekly runs; and `reddit_silver` has 2 manual runs — all with zero failures.

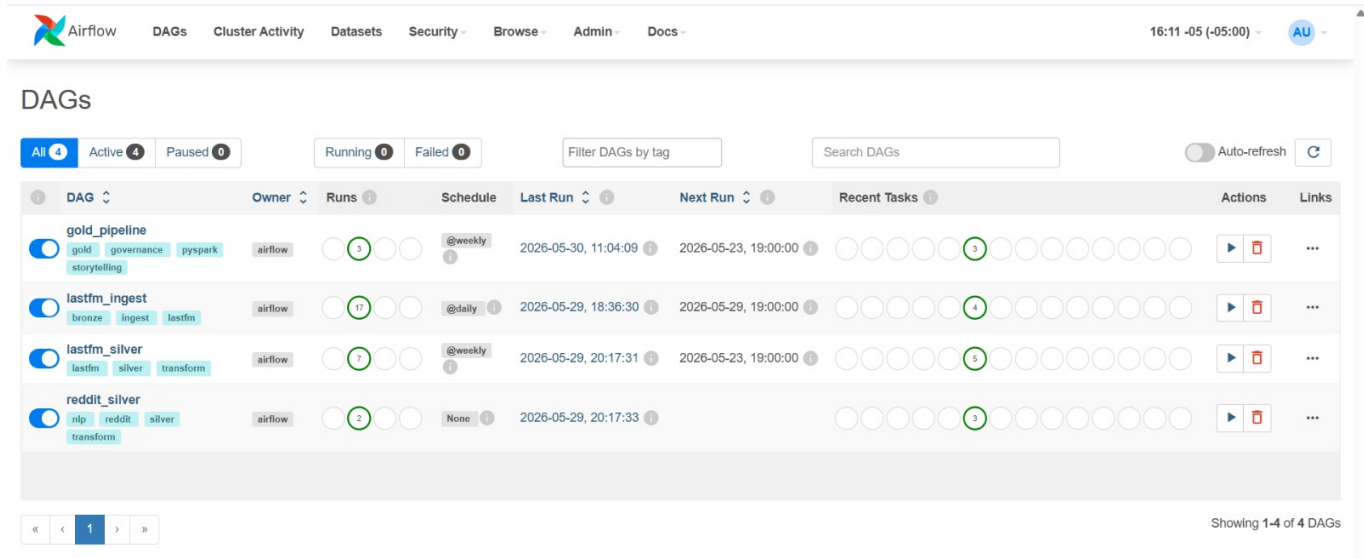


Figure 2: Airflow UI (`localhost:8080`) showing all four active DAGs with successful run histories. From top to bottom: `gold_pipeline` (@weekly, 3 runs), `lastfm_ingest` (@daily, 17 runs), `lastfm_silver` (@weekly, 7 runs), and `reddit_silver` (manual, 2 runs). Zero failures across all pipelines at time of submission.

Table 9 summarises the DAG execution status at the time of this submission.

Table 9: DAG execution status at submission

DAG	Schedule	Layer	Status
lastfm_ingest	@daily	Bronze	Active — 17 successful runs (last: 2026-05-29)
lastfm_silver	@weekly	Silver	Active — 7 successful runs (last: 2026-05-29)
reddit_silver	Manual	Silver	Active — 2 successful runs (last: 2026-05-29)
gold_pipeline	@weekly	Gold	Active — 3 successful runs, partitioned output (last: 2026-05-30)

6.5 Dashboard Screenshots

Figures 3–6 present full-page screenshots of both dashboards populated with real Gold data from the 2026-05-30 pipeline run.

Governance Dashboard (localhost:8050)

Figure 3 shows the main charts section. The four KPI cards confirm 228 Reddit comments processed, 150 Last.fm Artist snapshots, 100.0% schema compliance, and 0.0% highest null rate on required fields. The *Null Rate per Field* chart shows all required fields at exactly 0%. The *Outlier Rate* chart identifies reddit.score as the highest outlier field at 18%, consistent with the viral-post skew documented in Workshop #3. The *Mean Text Length* chart confirms that reddit.raw_comment (155 chars) is reduced to reddit.clean_comment (149 chars) by the NLP cleaning pipeline.

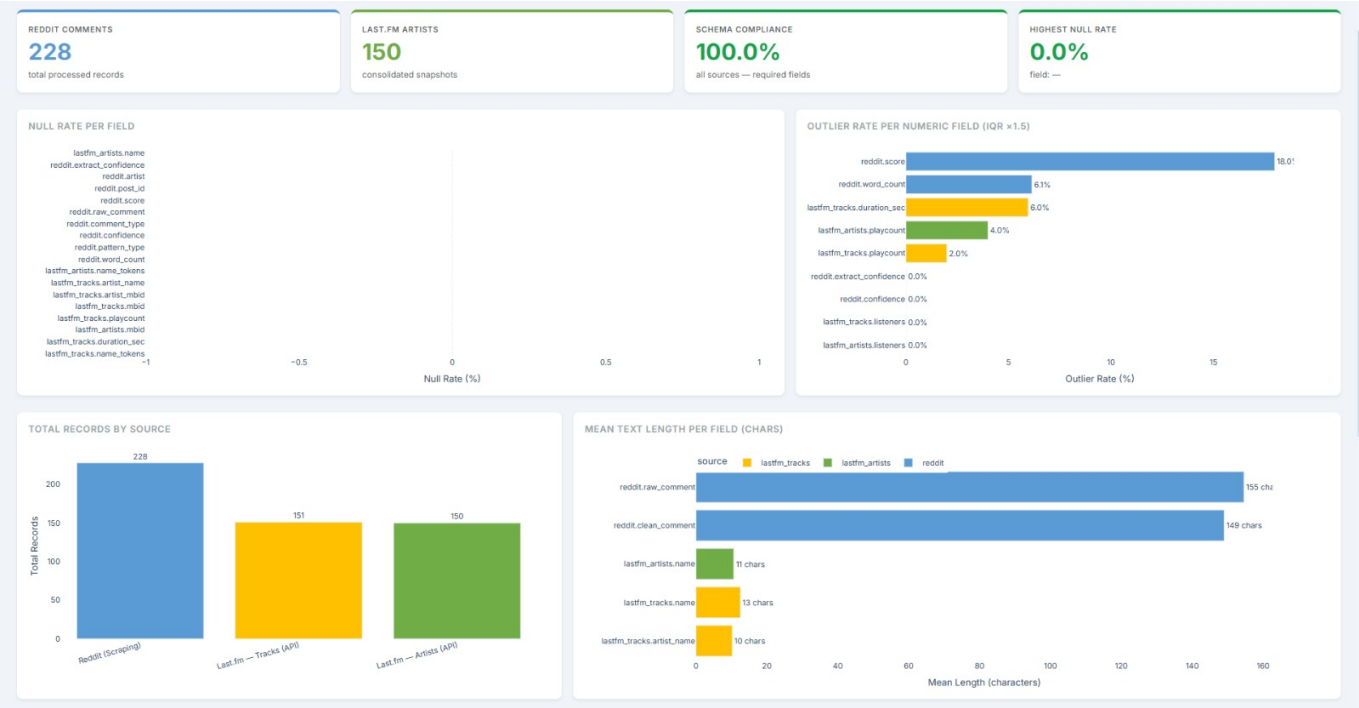


Figure 3: Governance Dashboard — KPI cards, null rate, outlier rate, records by source, and mean text length (localhost:8050).

Figure 4 shows the complete KPI table. The highlighted row (lastfm_artists / ALL / schema_compliance = 100) renders in green bold, confirming the conditional formatting rule fires correctly. The ingest_date column surfaces 2026-05-30 for all current-run rows. The table spans 5 pages covering all KPI types across the three sources.

COMPLETE DATA QUALITY KPI TABLE

\$SOURCE	\$FIELD	\$KPI	\$VALUE	\$UNIT	\$INGEST DATE
reddit	ALL	volume	228	count	2026-05-30
lastfm_tracks	name_tokens	null_rate	0	percentage	2026-05-30
lastfm_tracks	name	null_rate	0	percentage	2026-05-30
lastfm_tracks	ALL	volume	151	count	2026-05-30
lastfm_artists	ingested_at	ingestion_days	3	days	2026-05-30
lastfm_artists	name	text_len_max	18	characters	2026-05-30
lastfm_artists	name	text_len_min	3	characters	2026-05-30
lastfm_tracks	duration_sec	null_rate	0	percentage	2026-05-30
lastfm_artists	name	text_len_median	10	characters	2026-05-30
lastfm_artists	listeners	outlier_rate	0	percentage	2026-05-30
lastfm_artists	playcount	outlier_rate	4	percentage	2026-05-30
lastfm_artists	ALL	schema_compliance	100	percentage	2026-05-30
lastfm_artists	ingested_at	null_rate	0	percentage	2026-05-30
lastfm_artists	mbid	null_rate	0	percentage	2026-05-30
lastfm_artists	listeners	null_rate	0	percentage	2026-05-30

Filter data...

<<

<

1

/

5

>

>>

Figure 4: Governance Dashboard — Complete Data Quality KPI Table, page 1 of 5 (localhost:8050).

Storytelling Dashboard (localhost:8051)

Figure 5 shows the upper section. The dynamically generated *Key Insight* card reads: “68.0% of community comments are positive (avg score +0.621), while only 14.0% are negative. The conversation revolves around ‘album’, ‘music’, ‘good’. On Last.fm, Coldplay leads with 9M+ unique listeners.” The three summary pills give the functional user an immediate headline. The *Community Sentiment* donut confirms the distribution across 228 comments (positive 68%, neutral 18%, negative 14%). The *Sentiment Trend Over Time* chart plots the single available data point (2026-05-30, compound score $\approx +0.35$) within the shaded positive zone; as additional scraping sessions accumulate this chart will reveal temporal evolution.

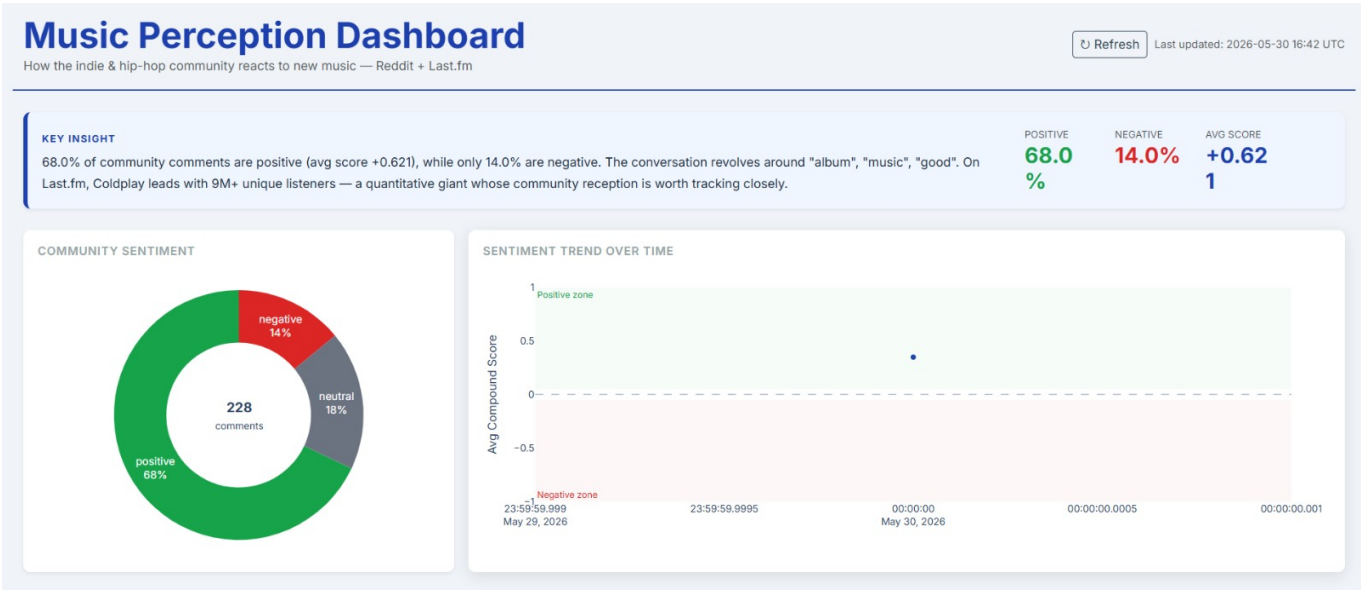


Figure 5: Storytelling Dashboard — narrative card, community sentiment donut, and sentiment trend over time (localhost:8051).

The lower section of the storytelling dashboard (Figure 6) covers conversation analysis and source comparison. The *Top Keywords* chart shows *album* (57 occurrences) as the dominant term, followed by *music* (41) and *good* (30); all rendered in green, confirming positive average sentiment. Terms such as *soundcloud* and *qobuz* survived the stop-word filter and reflect platform-discussion context within the communities. The *Comment Type Breakdown* confirms 78.5% opinion-type comments, 21.0% other, and 0.4% mixed — justifying the pivot to sentiment analysis as the primary analytical output. The *Volume Activity Over Time* grouped bar chart shows three ingestion dates (May 28–30), with Reddit (blue) appearing only on May 30 and Last.fm (green/amber) present on all three days. Finally, the *Top Artists* chart places Coldplay first with approximately 18M unique listeners, followed by Rihanna, Radiohead, Kanye West, and Lady Gaga.

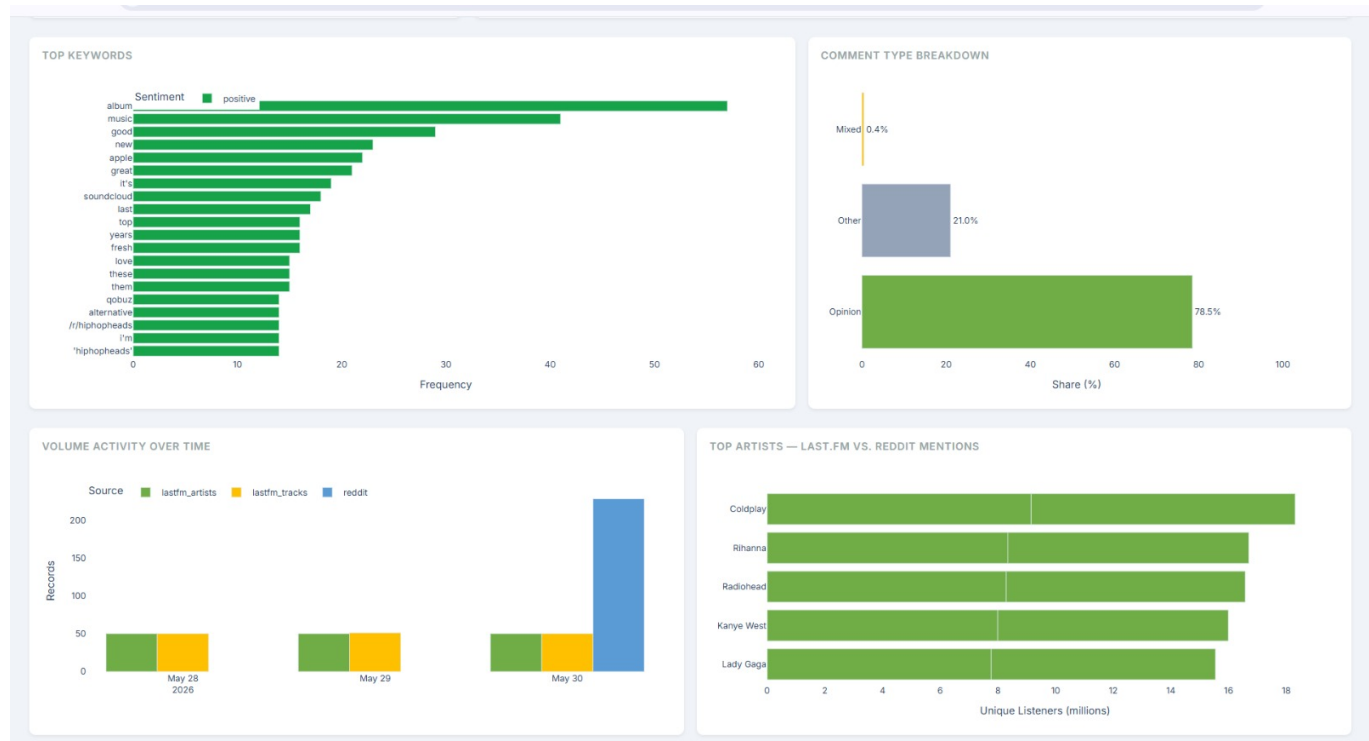


Figure 6: Storytelling Dashboard — lower section: top keywords, comment type breakdown, volume activity over time, and Last.fm vs. Reddit artist comparison.

7 Challenges and Resolutions

7.1 Challenge 1 — Gold File Path Mismatch Between DAG and Dashboards

Description

After introducing monthly partitioning in the Gold DAG, the dashboards initially failed to find any Parquet files because they were still looking for flat files at the root of `datalake_gold/` using the Workshop #3 path convention.

Resolution

The `_load_gold_dataset()` function in `utils.py` was updated to use a recursive glob pattern (`year=*/month=*/<name>.parquet`) that traverses the full partition tree. A backward-compatibility branch was added to handle CSV fallback files created in the Workshop #3 notebooks, ensuring development environments without a running Gold pipeline can still render the dashboards.

7.2 Challenge 2 — Schema Mismatch: `ingest_date` Field Not Present in Existing Parquets

Description

When the Gold DAG was re-run after adding `ingest_date` to both schemas, the append logic in `_save_parquet()` attempted to concatenate the new schema with a previously written Parquet that lacked the field, raising a `pyarrow.lib.ArrowInvalid` exception.

Resolution

Existing Gold Parquet files were deleted to force a clean first write with the new schema. The `_load_gold_dataset()` function was also updated with a rename guard (`data_date` → `ingest_date`) to handle any residual files written under an earlier field name during development.

7.3 Challenge 3 — Callback Firing on Every Interval Tick

Description

Both dashboards initially re-loaded all Gold Parquet files and rebuilt all figures on every 15-second interval tick, causing visible flickering and unnecessarily high CPU usage inside the Docker network.

Resolution

A `dcc.Store` component was added to each dashboard to persist a *state fingerprint* of the Gold folder between renders. The `gold_state_key()` function in `utils.py` computes this fingerprint by joining the relative paths and modification timestamps of all Parquet files under `datalake_gold/`. The callback compares the current fingerprint against the stored one and returns `dash.no_update` for all outputs if the files have not changed, eliminating unnecessary re-renders.

7.4 Challenge 4 — Keyword Noise from Bot Accounts and Platform Names

Description

Early runs of the storytelling dashboard showed high-frequency keywords such as *spotify*, *youtube*, *bot*, *links*, and *http* dominating the top-keyword chart. These tokens reflect platform-generated Reddit comments (music recommendation bots that post Spotify/YouTube links automatically) rather than genuine community discussion.

Resolution

A domain-specific stop-word extension was added to the `STOP_WORDS` set in `dag_gold.py`:

Listing 5: Domain-specific stop-word extension in `dag_gold.py`

```
1 STOP_WORDS = {  
2     # Standard English stop-words ...  
3     # Domain-specific noise detected in Reddit music communities  
4     "links", "bot", "message", "send", "youtube", "deezer", "spotify",  
5     "http", "www", "com", "amp", "gt", "lt", "via", "here", "check",
```

```
6 "listen", "song", "songs", "track", "tracks", "playlist", "link",  
7 }
```

After the Gold pipeline was re-run with the updated stop-word list, the top-keyword chart correctly surfaces substantive terms such as *album*, *music*, *good*, *love*, and *great* — terms that carry genuine analytical value for the functional user.